TERM II PROJECT REPORT

Library Collection Management System

Name: Ankita Kundu
Enrollment No.: 12024052004071
Roll No.: 74
Department: IT
Subject: Advanced Programming
Topic: DAO Pattern + File Persistence
Institution: Institute Of Engineering And Management

Problem Statement

Libraries managing their book collections manually or through unstructured digital methods face significant challenges in maintaining accurate and up-to-date records. Librarians often struggle with tracking book availability, monitoring stock levels, and recording borrowing and returning transactions efficiently. The absence of a structured system leads to data inconsistencies, difficulty in locating specific books, and the inability to identify low-stock titles that require restocking.

Furthermore, without persistent data storage, any recorded information risks being lost upon system shutdown, making reliable record-keeping nearly impossible. This project aims to develop a desktop-based Library Collection Management System using Java Swing, implementing the DAO Pattern and File Persistence to provide a structured, reliable, and maintainable solution for managing library book inventory, borrowing transactions, and collection records.

Target User

- School and college librarians managing daily book transactions
- Library assistants tracking borrowing and returning of books
- Small community or organization libraries without enterprise software
- CS/IT students learning DAO Pattern and File Persistence in Java

Core Features

- Add, edit, and delete books from the collection
- Borrow a book — decreases available quantity by 1
- Return a book — increases available quantity by 1
- Prevents borrowing when no copies are available
- Search books by title, author, or ISBN
- Filter books by genre or availability
- Low stock alert for books with copies at or below threshold
- Auto-saves data to books.dat after every operation
- Auto-loads data on application startup — no data loss on close
- Input validation with error messages for invalid entries
- Confirmation dialogs before delete and borrow operations

OOP Concepts Used

Abstraction

BookDAO interface abstracts all data access operations. The UI and Service layers only know what operations exist, not how they are implemented.

Encapsulation

Book.java keeps all fields private with public getters and setters. Each layer hides its internal logic from other layers.

Polymorphism

BookDAO dao = new BookDAOImpl() — the variable is declared as the interface type while the object is the concrete implementation, allowing storage to be swapped without changing other layers.

Inheritance

UI classes extend Swing components — MainFrame extends JFrame, BookTablePanel extends JPanel — enabling custom behavior on top of built-in Swing functionality.

Exception Handling

Handles FileNotFoundException on first launch, ClassNotFoundException during deserialization, and business rule violations such as borrowing when no copies are available or entering a negative quantity.

Collections

List<Book> and ArrayList store all books in memory. Java Streams are used for searching, filtering, and low stock queries across the collection.

Design Pattern

DAO Pattern is the central design pattern — isolating all file read and write operations inside BookDAOImpl, keeping the UI and Service layers independent of the data source.

Proposed Architecture Description

The system follows a 4-layer architecture based on the DAO Pattern where each layer has a single responsibility and communicates only with the layer directly below it.

Layer	Class / File	Responsibility
UI Layer	MainFrame, BookTablePanel, BookFormPanel, SearchPanel	Handles user input and display only. Never directly accesses files or data.
Service Layer	BookService	Validates input and enforces business rules such as preventing borrows when no copies are available.

Layer	Class / File	Responsibility
DAO Layer	BookDAO, BookDAOImpl	Defines and implements all data operations. Manages in-memory List<Book> and all file read/write.
Persistence Layer	books.dat	Binary file on disk. Written using ObjectOutputStream and read back using ObjectInputStream on every launch.

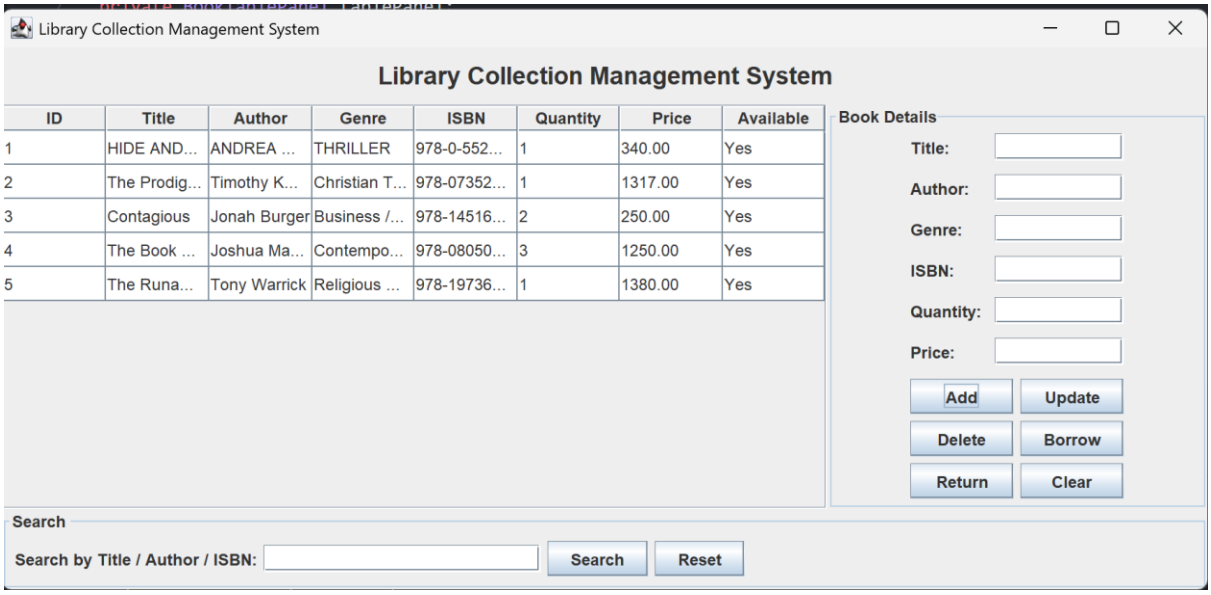
Every user action flows top to bottom through the layers — and every data refresh flows bottom to top back to the UI.

How to Run

Prerequisites

- JDK 17 or above installed
- Any Java IDE (IntelliJ IDEA, VS Code with Java Extension Pack)

Steps

- Clone the repository from GitHub
- Open the project in your IDE
- Navigate to the src folder
- Compile all Java files: cd src then javac *.java
- Run the application: java Main

Output